LQR Applied to a Non-Isothermal CSTR

Frank Vilimek

February 2026

1 CSTR Model

A Continuously-Stirred Tank Reactor (CSTR) is a type of continuous reactor that is commonly studied for process control due to its ease of modeling, yet flexibility to be expanded into complex setups and reaction networks. The benchmark for many research papers in process control for chemical engineering applications is the non-isothermal CSTR that is carrying out an exothermic reaction, a nonlinear process.

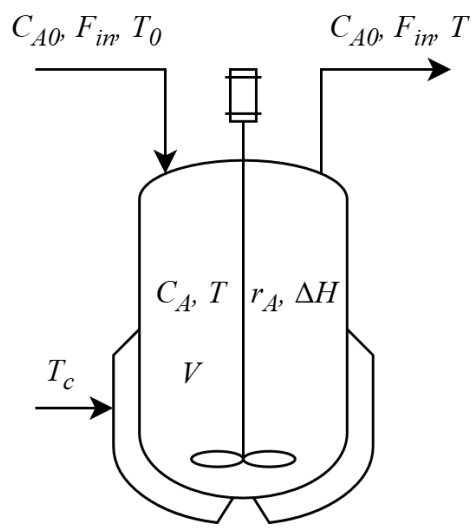

Figure 1: Non-isothermal CSTR with some relevant model parameters.

The model for the non-isothermal CSTR is given by the set of coupled ODEs

$$\frac{dC_A}{dt} = \frac{F_{in}}{V}(C_{A0} - C_A) - A_0 e^{-E/RT} C_A \tag{1}$$

$$\frac{dT}{dt} = \frac{F_{in}}{V}(T_0 - T) + \frac{(-\Delta H)}{\rho C_p V} A_0 e^{-E/RT} C_A - \frac{UA}{\rho C_p V}(T - T_c) \tag{2}$$

where the temperature-dependent reaction is commonly modeled as $A \rightarrow B$ (thus giving a first-order reaction rate, which also plays a role in making the system nonlinear). The variables and parameters in these equations are described in table (1).

Because there are multiple ways that the reactor dynamics can change (manipulating any of the inputs to the system, such as C_{A0} or T_0, will change the dynamics), the non-isothermal CSTR truly does give a bit

Table 1: Table of variable/parameter descriptions for the non-isothermal CSTR model.

Symbol	Description
C_A	Reactor and outlet stream concentration of species A
C_{A0}	Inlet concentration of species A
F_{in}	Inlet volumetric flow rate of reactant mixture
V	Volume of the CSTR
A_0	Frequency factor for the Arrhenius reaction rate
E	Activation energy for the reaction
R	Ideal gas constant
T	Reactor and outlet stream temperature
T_0	Inlet temperature of reactant mixture
T_c	Temperature of cooling fluid
ΔH	Enthalpy of reaction
ρ	Mass density of fluid mixture
C_p	Constant-pressure heat capacity
U	Overall heat transfer coefficient of heating jacket
A	Heat transfer area of heating jacket

more freedom when applying control methods. In order to apply control, we first need a way to represent our model in a more standard form.

2 State-Space Representation

The state-space representation of a dynamic model is a standard form for understanding the dynamics of a system based on its current state and a number of inputs. The state-space representation for any model is

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \tag{3}$$

$$\mathbf{y} = C\mathbf{x} + D\mathbf{u} \tag{4}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector, $\dot{\mathbf{x}} \in \mathbb{R}^n$ is the time rate of change of the state vector, $\mathbf{u} \in \mathbb{R}^m$ is the control vector, $\mathbf{y} \in \mathbb{R}^p$ is the output vector, $A \in \mathbb{R}^{n\times n}$ is the state matrix, $B \in \mathbb{R}^{n\times m}$ is the input matrix, $C \in \mathbb{R}^{p\times n}$ is the output matrix, and $D \in \mathbb{R}^{p\times m}$ is the feedthrough matrix (D is often set to zero).

The state-space representation of a model allows us to separate state-dependent dynamics (any terms that multiply a state variable in $\mathbf{x}$) from control-dependent dynamics (any terms that multiply a potentially manipulated variable in a chosen $\mathbf{u}$). This gives us the flexibility to more-or-less swap out manipulated variables $\mathbf{u}$, so long as we know how the system behaves on its own as well as how it will react given a certain input. It should be emphasized that $\mathbf{u}$ is not a requirement for having an accurate model, not having $\mathbf{u}$ just means that the system will evolve however it would without additional inputs.

Developing the state-space representation of a dynamic model requires having expressions for the time

rate of change of desired variables (for example, C_A and T), which may require linearization around a steady-state point if terms are nonlinear. Once our model is in state-space form, we are ready to implement control. In this document, we are interested in implementing a linear-quadratic regulator (LQR), a form of optimal control, and seeing its performance in controlling a nonlinear system such as our CSTR.

3 Linear-Quadratic Regulator (LQR)

The linear-quadratic regulator (LQR) is a form of optimal control that places penalties on deviations from a setpoint for the state variables as well as penalties on control effort—it ensures that control is applied such that state deviations and control effort are simultaneously minimized.

At the heart of LQR is the cost function, an expression that quantifies the "cost" of a certain trajectory or path through time. These cost functions, which all calculate the square of the deviations and control effort (where the "quadratic" part of LQR comes from), come in a few forms; an easy to understand form is the infinite-horizon, continuous-time cost:

$$J = \int_0^\infty \left(\mathbf{x}^\top Q\mathbf{x} + \mathbf{u}^\top R\mathbf{u}\right) dt \tag{5}$$

where $Q \in \mathbb{R}^{n\times n}$ is the state cost matrix and $R \in \mathbb{R}^{m\times m}$ is the control cost matrix. The choice of Q and R is up to the discretion of the control system designer, but must be chosen wisely so that the controlled system meets performance metrics.

The control vector $\mathbf{u}$ is what is calculated such that J is minimized. Most commonly, a linear controller (where the "linear" part of LQR comes from) is implemented with the control law

$$\mathbf{u} = -K\mathbf{x} \tag{6}$$

where K is the gain matrix. K is defined such that any values of $\mathbf{u}$ calculated using this control law guarantees that the cost is minimized. The expression for K is

$$K = R^{-1}B^\top P \tag{7}$$

where R is the same control cost matrix in the cost function, B is the input matrix in the state-space representation, and P is the solution to the Continuous-Time Algebraic Riccati Equation (CARE)

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0 \tag{8}$$

where Q is the same state cost matrix from the cost function, and A is the state matrix in the state-space representation. Scipy has the function `scipy.linalg.solve_continuous_are()` that solves the CARE for us, making our lives much easier.

LQR is guaranteed to drive our system towards the origin (where the "regulator" part of LQR comes from), given that the model and its implementation are correct. We can slightly modify our model using the error between the state and a setpoint instead of the state itself, which grants us the ability of setpoint tracking. This most typically takes the form of defining deviation variables, but other methods such as augmenting the state vector with an extra term for the integral of error are also done.

4 Building the Model

We are now ready to put our model described by equations (1) and (2) into state-space form and apply LQR to it. There are a few steps we need to take before we get our final result:

1. Find steady-state values for C_A, T, and our choice of manipulated variable at given setpoints
2. Linearize nonlinear terms in the rate of change equations for C_A and T
3. Define deviation variables for C_A, T, and the choice of manipulated variable relative to the setpoint values
4. Put the linearized equations into state-space form
5. Formulate the cost function and calculate the optimal gain matrix
6. Simulate the model with LQR control

4.1 Steady-State Values at the Setpoint

We first need to decide what our setpoint will be, which acts as the anchor for the steady-state values of our state variables. Let us arbitrarily choose a setpoint temperature T^{ss} (it could have easily been for concentration, flowrate, coolant temperature, etc.)

Steady-state occurs when the time derivative is set to zero. Doing this, we can solve for the steady-state values of C_A and T at a given T^{ss}:

$$C_A^{ss} = \frac{F_{in} C_{A0}}{F_{in} + V A_0 e^{-E/RT^{ss}}} \tag{9}$$

$$T^{ss} = \left[\frac{F_{in}}{V} + \frac{UA}{\rho C_p V}\right]^{-1} \left[\frac{F_{in}}{V} T_0 + \frac{UA}{\rho C_p V} T_c - \frac{F_{in} C_{A0} \Delta H e^{-E/RT^{ss}}}{\rho C_p \left(F_{in} + V A_0 e^{-E/RT^{ss}}\right)}\right] \tag{10}$$

The value for T^{ss} will need to be solved numerically, using `scipy.optimize.fsolve()` for example. This only needs to be done once at the start of the calculation/simulation.

It is important to find the steady-state values prior to linearization because the linearized model is not guaranteed to capture the nonlinear properties of the actual model as the system moves away from the steady-state point.

4.2 Linearization of Nonlinear Terms

We then need to linearize any nonlinear terms (in our case, the reaction rate constant k and any terms multiplying it) around the steady-state values we calculated in the previous step. We do this through a Taylor series expansion up to the linear term. As a reminder, the Taylor series expansion for a multivariate function is:

$$f(x, y) \approx f(x_0, y_0) + \left.\frac{\partial f}{\partial x}\right|_{x_0, y_0} (x - x_0) + \left.\frac{\partial f}{\partial y}\right|_{x_0, y_0} (y - y_0) + \dots \tag{11}$$

Thus, the reaction rate r_A is linearly approximated as

$$r_A = kC_A \approx k^{ss} C_A^{ss} + k^{ss}(C_A - C_A^{ss}) - \frac{Ek^{ss}}{RT^{ss2}}(T - T^{ss}) \tag{12}$$

This can then be used in equations (1) and (2) to make them linearized around their steady-states determined by our setpoint:

$$\frac{dC_A}{dt} = \frac{F_{in}}{V}(C_{A0} - C_A) - \left[k^{ss}C_A^{ss} + k^{ss}(C_A - C_A^{ss}) - \frac{Ek^{ss}}{RT^{ss2}}(T - T^{ss})\right] \tag{13}$$

$$\frac{dT}{dt} = \frac{F_{in}}{V}(T_0 - T) - \frac{\Delta H}{\rho C_p}\left[k^{ss}C_A^{ss} + k^{ss}(C_A - C_A^{ss}) - \frac{Ek^{ss}}{RT^{ss2}}(T - T^{ss})\right] + \frac{UA}{\rho C_p V}(T_c - T) \tag{14}$$

4.3 Deviation Variables

We now need to define deviation variables, which are variable substitutions that act to shift the range of values to be relative to a steady-state value—in other words, they allow us to ignore any constants that are not relevant to the dynamics of the system, such as constant offsets. We define the following deviation variables:

$$C_A' = C_A - C_A^{ss}, \qquad T' = T - T^{ss}, \qquad T_c' = T_c - T_c^{ss}. \tag{15}$$

Substituting these deviation variables into equations (13) and (14) yields the following models:

$$\frac{dC_A'}{dt} = -\left[\frac{F_{in}}{V} + k^{ss}\right]C_A' + \left[\frac{Ek^{ss}}{RT^{ss2}}\right]T' \tag{16}$$

$$\frac{dT'}{dt} = -\left[\frac{\Delta H k^{ss}}{\rho C_p}\right]C_A' + \left[\frac{EK^{ss}}{RT^{ss2}} - \frac{F_{in}}{V} - \frac{UA}{\rho C_p V}\right]T' + \left[\frac{UA}{\rho C_p V}\right]T_c' \tag{17}$$

4.4 Linearized Equations into State-Space

The models are now able to be put into state-space representation. Since the coefficients for each state variable and control variable are already factored out, our job here is quite easy:

$$\underbrace{\begin{bmatrix} \frac{dC_A'}{dt} \\ \frac{dT'}{dt} \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} -\left(\frac{F_{in}}{V} + k^{ss}\right) & \frac{Ek^{ss}}{RT^{ss2}} \\ -\frac{\Delta H k^{ss}}{\rho C_p} & \left(\frac{Ek^{ss}}{RT^{ss2}} - \frac{F_{in}}{V} - \frac{UA}{\rho C_p V}\right) \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} C_A' \\ T' \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 \\ \frac{UA}{\rho C_p V} \end{bmatrix}}_{B} \underbrace{T_c'}_{\mathbf{u}} \tag{18}$$

We can easily see how the state space model maps to the general form we saw earlier in equation (3). What you may notice when you model different systems is that the control matrix B and control vector $\mathbf{u}$ will change depending on what you choose to control. This means that, essentially, B and $\mathbf{u}$ can be interchanged with whatever you choose, so long as you linearized the model properly and have expressions that describe how changes in your chosen control variable will affect the system.

4.5 Cost Function and Optimal Gain

The last step before we model the system is defining the state cost and control cost matrices Q and R. We can really choose whatever values they contain, so long as they are of the proper dimension ($Q \in \mathbb{R}^{2\times 2}$, $R \in \mathbb{R}^{1\times 1}$):

$$Q = \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix}, \qquad R = r \tag{19}$$

To make calculations faster, Q and R can be chosen to be diagonal, but it is not required. We can use these Q and R matrices to then calculate the optimal gain K using equation (7).

4.6 Simulate the Model with LQR Control

We can now finally model our system with LQR control. Example Python code for the calculations are below, with example results in figure (2).

Non-Isothermal CSTR LQR Control Python Code

```
# Import libraries
import numpy             as np
import matplotlib.pyplot as plt
import scipy             as sc

# Initialize constants
Cf, F0, V, rho, H, Cp, Cpj, UA, Tc0, Tf, A0, E, R = [
    10, 5, 10, 10e3, -10e4, 4.184, 1, 100, 200, 500, 10e5, 4 * 10e4, 8.314
]

# Simulation parameters
totalTime = 30
dt        = 0.01
timeSim   = np.arange(0, totalTime, dt)

# T_ss will be the setpoint, calculate CA_ss and k_ss
T_ss  = 450
CA_ss = F0 * Cf / (F0 + V * A0 * np.exp(-E / (R * T_ss)))
k_ss  = A0 * np.exp(-E / (R * T_ss))

# Define initial conditions
CA0 = 0
T0  = 100

# Initialize arrays of CA, T, and Tc, then set initial conditions
CA = np.zeros(len(timeSim))
T  = np.zeros(len(timeSim))
Tc = np.zeros(len(timeSim))
u  = np.zeros(len(timeSim))

CA[0] = CA0
T[0]  = T0

# Define Am, B, Q, and Rm matrices
Am = np.array([
    [-(F/V + k_steadystate), E * k_steadystate / (R * T_steadystate**2)],
    [H * k_steadystate / (rho * Cp), E * k_steadystate / (R * T_steadystate**2) - F/V - U *
     A / (rho * Cp * V)],
])

B = np.array([
    [0],
```

```
    [U * A / (rho * Cp * V)],
])

Q = np.array([
    [10000, 0],
    [0, 10000],
])

Rm = np.array([
    [1],
])

# Solve algebraic Riccati equation
P = sc.linalg.solve_continuous_are(Am, B, Q, Rm)
K = sc.linalg.inv(Rm) @ B.T @ P

# Carry out simulation
for t in range(len(time) - 1):

    # Arrhenius reaction rate
    k_t = A0 * np.exp(-E / (R * T[t]))

    # Calculate control action based on current state
    state = np.array([
        [CA[t] - CA_steadystate],
        [T[t] - T_steadystate],
    ])

    u_step    = -K @ state
    u[t + 1]  = u_step[0][0]
    Tc[t + 1] = Tc[t] + (dt * u_step[0][0]) / Cpj

    # Finite difference simulation
    CA[t + 1] = CA[t] + dt * (F/V * (Cf - CA[t]) - k_t * CA[t])
    T[t + 1]  = T[t] + dt * (F/V * (Tf - T[t]) + H/(rho * Cp) * k_t * CA[t] + U * A / (rho
    * Cp * V) * (Tc[t] - T[t]))
```

After you run this program, plotting the results should yield the graphs seen in figure (2).

5 Model Behavior and Analysis

We can now see what our LQR controller does to a nonlinear such as our non-isothermal CSTR. We see that since the steady-state concentration has been set around the temperature setpoint, as the controller drives the jacket temperature to raise/reduce the reaction temperature towards the setpoint, the concentration follows in moving towards its setpoint.

The behavior of the reaction rate over time in relation to the reactor temperature also matches our intuition—as concentration increases, the reaction rate increases, and so does the temperature (which is also increased by the controller). The increased temperature drives a faster reaction, causing the concentration to drop significantly to nearly zero, where the reaction rate slows. The slowed reaction rate can now be

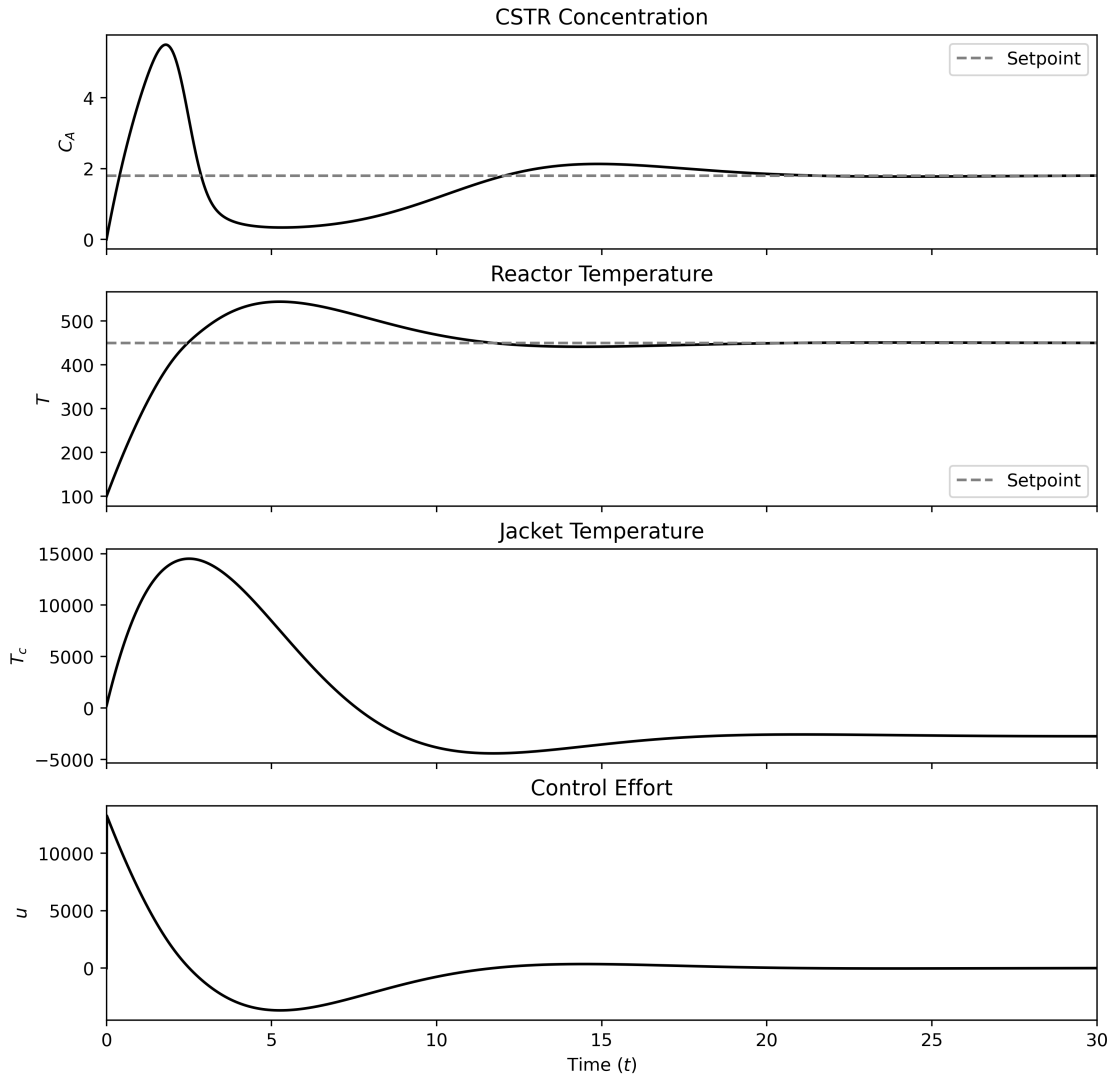

Figure 2: LQR control for a non-isothermal CSTR with an exothermic reaction.

controlled easier, and the control effort also levels off.

It is beneficial to keep in mind that LQR is an optimal controller with *no constraints*, so it is entirely possible that some of the values are not realistic (for example, the control effort, which in our case is the jacket temperature in Kelvin, goes negative). In order to model constraints, we will either need to make use of constrained LQR or, the more popular option, model-predictive control (MPC).